Guía completa del proyecto Tres en Raya

Minimax, Backtracking y Árbol de Decisión (ML) con patrón MVC

Alessandro Callejas Calizaya — Estructura de Datos 2

Índice

1. Qué hace el proyecto **2**

2. ¿Cumple con el PDF? Qué falta y qué sobraba **2**
2.1 Verificación contra la propuesta . . . 2
2.2 Qué se eliminó o simplificó (lo que el PDF no pedía) . . . 3
2.3 Hallazgos importantes (conviene leerlos antes de entregar) . . . 4

3. Arquitectura MVC **5**
3.1 El viaje de un clic . . . 6

4. Guía de cada archivo, clase y método **6**
4.1 `model/game_model.py` — `TresEnRayaModel` . . . 6
4.2 `model/minimax_agent.py` — `MinimaxAgent` . . . 6
4.3 `model/ml_agent.py` — `MLAgent` . . . 7
4.4 `view/gui_view.py` — `TresEnRayaView` . . . 8
4.5 `controller/game_controller.py` — `GameController` . . . 8
4.6 `main.py` y `comparativa_rendimiento.py` . . . 8

5. Minimax y backtracking en pocas líneas **8**

6. Cómo se probó y qué quedó pendiente de tu lado **10**

7. Tabla de nombres cambiados **10**

1. Qué hace el proyecto

Es un juego de **Tres en Raya** con interfaz Tkinter que se puede jugar en tres modos:

1. **Humano vs Humano.**
2. **Humano vs IA Minimax:** la IA recorre el árbol de jugadas con *backtracking* y el panel muestra cuántos nodos exploró y cuánto tardó.
3. **Humano vs IA Machine Learning:** un `DecisionTreeClassifier` (Scikit-Learn), entrenado con jugadas que Minimax etiquetó, predice la jugada sin explorar el árbol.

Está organizado con el patrón **Modelo-Vista-Controlador** (carpetas `model/`, `view/`, `controller/`), tal como pide el PDF de la propuesta.

Cómo ejecutarlo (desde la carpeta del proyecto):

```
python main.py                        # abre el juego
python comparativa_rendimiento.py     # compara Minimax contra el árbol ML
python -m model.ml_agent              # regenera reglas y gráfico del árbol
```

Requiere Python 3 con Tkinter, `pandas`, `scikit-learn` y `matplotlib`.

2. ¿Cumple con el PDF? Qué falta y qué sobraba

2.1 Verificación contra la propuesta

Requisito del PDF	Estado	Dónde / evidencia
Estructura MVC con los 5 archivos indicados	Cumple	`model/` (3 archivos), `view/gui_view.py`, `controller/game_controller.py`
Sin lógica de juego en la vista	Cumple	`gui_view.py` solo dibuja y avisa por callbacks
Tablero 3x3, movimientos válidos, victoria, empate	Cumple	`TresEnRayaModel`
Backtracking: aplicar y deshacer movimientos	Cumple	`make_move` / `undo_move`
Función de evaluación +10 / -10 / 0	Cumple	`MinimaxAgent.evaluate`
Minimax recursivo con contador de nodos	Cumple	igual al código de referencia del PDF (sección 5)
Agente imbatible	Cumple (probado)	Minimax enfrentó a X probando **todas** sus jugadas posibles: 569 partidas, X nunca ganó
Backtracking restaura el estado sin errores	Cumple (probado)	el tablero quedó idéntico tras cada `get_best_move`
Selector de 3 modos y métricas (nodos y tiempo)	Cumple	`gui_view.py` + `game_controller.py`
Dataset de 1,000+ partidas simuladas	Cumple	`generar_dataset_sintetico(1000)`; el CSV tiene 532 tableros únicos
Limpieza con Pandas y 9 características + etiqueta	Cumple	`cargar_y_limpiar_dataset`, columnas `c0..c8` y `jugada_optima`
`DecisionTreeClassifier` entrenado	Cumple	`entrenar_arbol`

Requisito del PDF	Estado	Dónde / evidencia
Visualización de reglas (`export_text`) y gráfico (`plot_tree`)	Cumple	`reglas_arbol_decision.txt`, `arbol_decision.png`
ML integrado en la interfaz	Cumple	modo “Humano vs IA ML (Árbol)”
Comparación Minimax vs ML (latencia y precisión)	Cumple, con una corrección	`comparativa_rendimiento.py` (ver 2.3)
Documento de análisis comparativo	Existe	`reporte_comparativo.md` (ver 2.3: conviene actualizar cifras)

Conclusión: el proyecto implementa las tres semanas del PDF. No falta ningún entregable; lo pendiente son ajustes de calidad que se explican abajo.

2.2 Qué se eliminó o simplificó (lo que el PDF no pedía)

Archivo	Se eliminó / simplificó	Motivo
`game_model.py`	4 métodos en español duplicados (`hacer_movimiento`, `deshacer_movimiento`, `obtener_movimientos_validos`, `verificar_victoria`) que solo llamaban a los métodos en inglés	Cada acción tenía dos nombres; el PDF usa `make_move`, `undo_move`, `get_available_moves`, `check_winner`
`game_model.py`	Tablero duplicado en `__init__` y `reiniciar`	Ahora `__init__` llama a `reiniciar()`
`ml_agent.py`	`obtener_resumen_dataset` (nadie lo usaba), `estructurar_caracteristicas_y_objetivo` (se fundió en `dividir_datos_entrenamiento`), parámetro `eliminar_duplicados`, atributo `accuracy`, `sys.path` manual, imports `typing`, docstrings de 15 líneas	Código muerto o no pedido
`ml_agent.py`	Se creaba un `MinimaxAgent` nuevo en **cada** jugada del rival simulado	Ahora se crea una sola vez (`minimax_rival`)
`ml_agent.py`	Bloque de demo de ~50 líneas	Reducido a 8 líneas que regeneran reglas y gráfico
`game_controller.py`	Import de `MLAgent` sin uso, valores por defecto `agente_ia=None`, chequeo de “ML no entrenado”, comprobaciones de `None` imposibles	`main.py` siempre entrega los agentes ya listos
`game_controller.py`	Código de fin de partida repetido 3 veces dentro de `manejar_clic`	Extraído a `_terminar_si_corresponde`

Archivo	Se eliminó / simplificó	Motivo
`gui_view.py`	Tres `Radiobutton` copiados y pegados	Un bucle sobre `opciones_modo` (mismos widgets, mismo diseño)
`comparativa_rendimiento.py`	`sys.path` manual, impresiones decorativas	Más corto y claro
Todos	Comentarios de "Semana X - Día Y" y analogías ("Restaurante")	Ya no aportan al leer el código

No se tocó: diseño de Tkinter (mismos widgets, tamaños y textos), `registro_proyecto.txt`, `reporte_comparativo.md`, el CSV, las reglas y la imagen del árbol. Tampoco se hizo ningún `git commit`.

2.3 Hallazgos importantes (conviene leerlos antes de entregar)

1. El benchmark original comparaba en tableros inválidos. `generar_estados_prueba` hacía entre 1 y 6 jugadas previas alternando X y O. Cuando eran 2, 4 o 6, le tocaba a **X**, pero el benchmark le pedía igual la jugada a la IA `0`. En una muestra de 300 tableros, 142 (47 %) eran de ese tipo. El árbol ML solo se entrenó con tableros donde mueve O, así que esa mitad no medía nada útil. Ya está corregido: ahora solo se generan tableros con 1, 3 o 5 fichas (donde sí mueve O).

2. Por eso las cifras de `reporte_comparativo.md` ya no coinciden con el script. En mi entorno (computador distinto al tuyo, los milisegundos no son comparables), el benchmark corregido da aproximadamente: ~25,000 nodos por jugada para Minimax (el reporte dice 16,531), coincidencia con Minimax del 63 % al 69 % (el reporte dice 58 %-65 %) y una aceleración de más de 100x. **Vuelve a ejecutar `python comparativa_rendimiento.py` en tu PC y actualiza la tabla del reporte con tus números.**

3. La explicación de la "precisión" del reporte es optimista. El reporte atribuye la diferencia a empates entre jugadas igual de buenas. Medí algo más estricto (300 tableros): el árbol ML elige una jugada con **el mismo valor que la de Minimax solo en ~76 %** de los casos. Es decir, en ~24 % elige una jugada peor, no solo equivalente. Esto es normal: el árbol aprendió de 532 tableros y su precisión en el conjunto de prueba es ~45 %. Para el reporte conviene decirlo así: *el árbol ML es mucho más rápido pero no es infalible; Minimax sí lo es*. Eso además es justo lo que el PDF pide comparar.

3. Arquitectura MVC

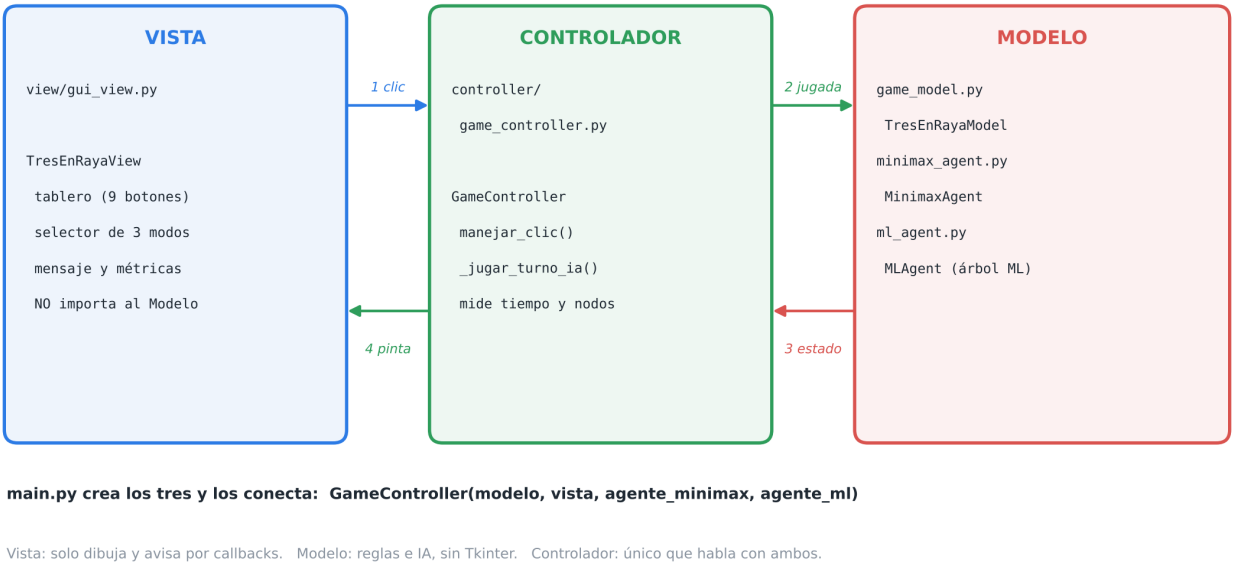


Figura 1: Las tres capas y cómo se conectan. La IA vive en el Modelo.

Capa	Archivo	Responsabilidad	Qué NO hace
Modelo	`model/game_model.py`	Tablero y reglas	No conoce Tkinter ni las IAs
Modelo	`model/minimax_agent.py`	IA por búsqueda (Minimax)	No toca la pantalla
Modelo	`model/ml_agent.py`	IA por aprendizaje (árbol de decisión)	No toca la pantalla
Vista	`view/gui_view.py`	Dibuja tablero, modos y métricas	No importa al Modelo ni decide jugadas
Controlador	`controller/game_controller.py`	Turnos, clics, pide jugadas a la IA, mide tiempo	No dibuja widgets
Arranque	`main.py`	Crea y conecta las piezas	—

3.1 El viaje de un clic

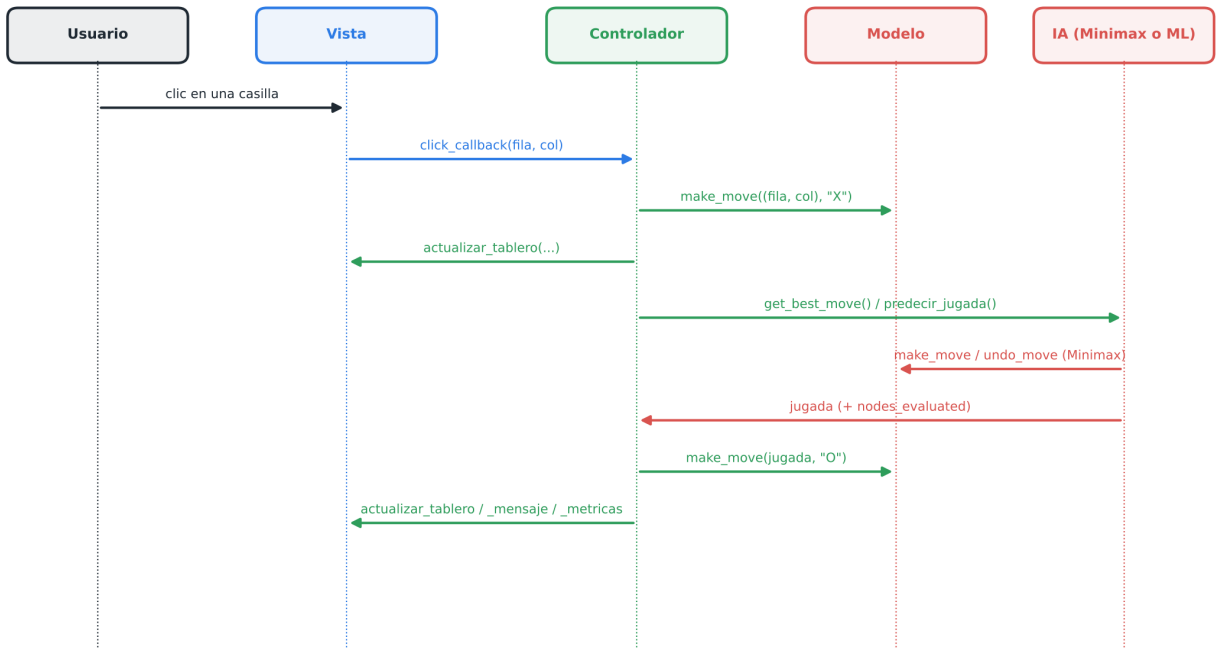


Figura 2: Desde el clic hasta que aparece la jugada de la IA.

1. El usuario pulsa una casilla; la Vista llama `click_callback(fila, col)`, que en realidad es `GameController.manejar_clic`.
2. El Controlador aplica la jugada en el Modelo con `make_move` y actualiza la Vista.
3. Si el modo es Minimax o ML, `_jugar_turno_ia` pide la jugada a la IA y mide cuánto tarda.
4. El Controlador aplica la jugada de la IA, actualiza tablero, mensaje y métricas.

4. Guía de cada archivo, clase y método

4.1 `model/game_model.py` — `TresEnRayaModel`

Guarda el tablero como una matriz 3x3 (`' '` = libre, `'X'` u `'O'`). Los cuatro nombres en inglés son los que usa el agente Minimax del PDF.

Método	Qué hace
`__init__()`	Crea el tablero vacío (llamando a `reiniciar`)
`reiniciar()`	Deja las 9 casillas libres
`get_available_moves()`	Lista de casillas libres como `(fila, col)`
`make_move((fila, col), jugador)`	Pone la ficha; devuelve `False` si estaba ocupada
`undo_move((fila, col))`	**Backtracking:** vacía la casilla
`check_winner(jugador)`	`True` si completó fila, columna o diagonal
`es_empate()`	`True` si no quedan casillas libres

4.2 `model/minimax_agent.py` — `MinimaxAgent`

Elemento	Qué hace
`ai_player`, `human_player`	Fichas de la IA (`O`) y del humano (`X`)
`nodes_evaluated`	Contador de nodos del árbol visitados
`evaluate(board)`	Valor de un estado final: +10 gana IA, -10 gana humano, 0 resto
`minimax(board, depth, is_maximizing)`	Recursión: la IA maximiza, el humano minimiza. Casos base: victoria (`±10` ajustado por `depth`) o empate (0)
`get_best_move(board)`	Prueba cada casilla libre con `minimax` y devuelve la de mayor puntaje (si empatan, la primera)

4.3 `model/ml_agent.py` — `MLAgent`

Elemento	Qué hace
`VALOR_FICHA`	Convierte fichas en números: vacía 0, `X` 1, `O` -1
`COLUMNAS_CASILLAS` / `COLUMNA_JUGADA`	`c0..c8` (entrada) y `jugada_optima` (salida)
`vectorizar_tablero(tablero)`	Tablero 3x3 -> lista de 9 números
`coordenadas_a_indice` / `indice_a_coordenadas`	`(fila, col)` <-> casilla 0..8
`simular_partida(...)`	Juega una partida automática; en cada turno de la IA guarda (tablero, mejor jugada según Minimax)
`generar_dataset_sintetico(num_partidas, ruta)`	Simula partidas, elimina tableros repetidos y guarda el CSV
`guardar_csv(ruta)`	Escribe los registros en CSV
`cargar_y_limpiar_dataset(ruta)`	Lee con Pandas; quita nulos, valores fuera de rango y duplicados
`dividir_datos_entrenamiento()`	Separa 9 características y la jugada; 80 % entrenamiento / 20 % prueba
`entrenar_arbol(criterion, max_depth)`	Entrena el `DecisionTreeClassifier` y devuelve su precisión
`predecir_jugada(tablero)`	Predice `(fila, col)`; si la casilla sugerida está ocupada, usa la libre más probable
`exportar_reglas_texto(ruta)`	Reglas “si… entonces…” con `export_text`
`exportar_arbol_grafico(ruta)`	Imagen del árbol con `plot_tree`

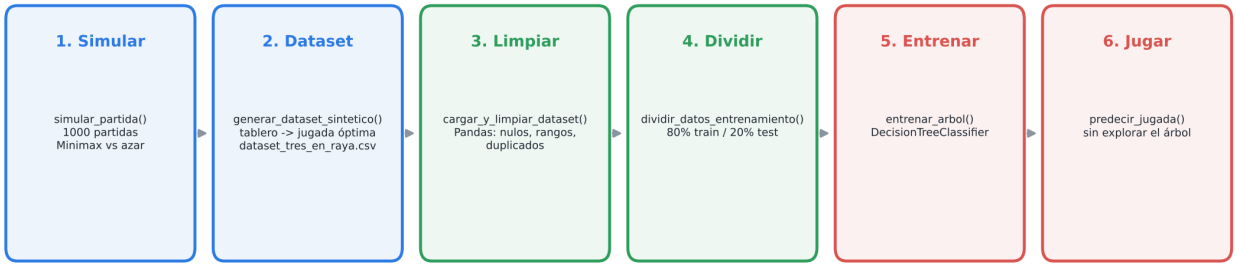


Figura 3: Pipeline de Machine Learning: de las partidas simuladas a la jugada predicha.

4.4 `view/gui_view.py` — `TresEnRayaView`

Elemento	Qué hace
`botones`	Matriz 3x3 de botones del tablero
`etiqueta_mensaje`	Turno o resultado
`modo_seleccionado`	Modo elegido en los radiobuttons
`etiqueta_nodos`, `etiqueta_tiempo`	Métricas de la IA
`set_click_callback` / `set_modo_callback` / `set_reiniciar_callback`	El Controlador registra aquí sus funciones
`_on_boton_click`, `_on_modo_cambiado`, `_on_reiniciar_click`	Reenvían el evento al Controlador
`actualizar_tablero`, `actualizar_mensaje`, `actualizar_metricas`	Cambian lo que se ve en pantalla
`refrescar()`	Fuerza el redibujado para mostrar "pensando…" antes de calcular

4.5 `controller/game_controller.py` — `GameController`

Método	Qué hace
`__init__(modelo, vista, agente_minimax, agente_ml)`	Guarda las piezas y registra los callbacks en la Vista
`manejar_clic(fila, col)`	Aplica la jugada, revisa fin de partida y pasa el turno o llama a la IA
`_jugar_turno_ia()`	Pide la jugada a Minimax o al árbol ML, mide tiempo y nodos, y la aplica
`_terminar_si_corresponde(mensaje)`	Detecta victoria o empate y cierra la partida
`_pasar_turno(ficha)`	Cambia el turno y lo muestra
`cambiar_modo(modo)`	Cambia de modo y reinicia
`reiniciar_juego()`	Tablero vacío y métricas en cero

4.6 `main.py` y `comparativa_rendimiento.py`

- **`main.py`:** `preparar_agente_ml()` entrena el árbol (genera el dataset si falta) y `main()` crea Modelo, Vista y Controlador y abre la ventana. Su docstring inicial resume el proyecto.
- **`comparativa_rendimiento.py`:** genera tableros válidos donde mueve O, mide tiempo y nodos de Minimax contra el árbol ML y calcula la coincidencia de jugadas.

5. Minimax y backtracking en pocas líneas

El árbol de juego **no se guarda en memoria**: la recursión de `minimax()` lo recorre. Cada llamada es un nodo y cada casilla libre probada es una rama.

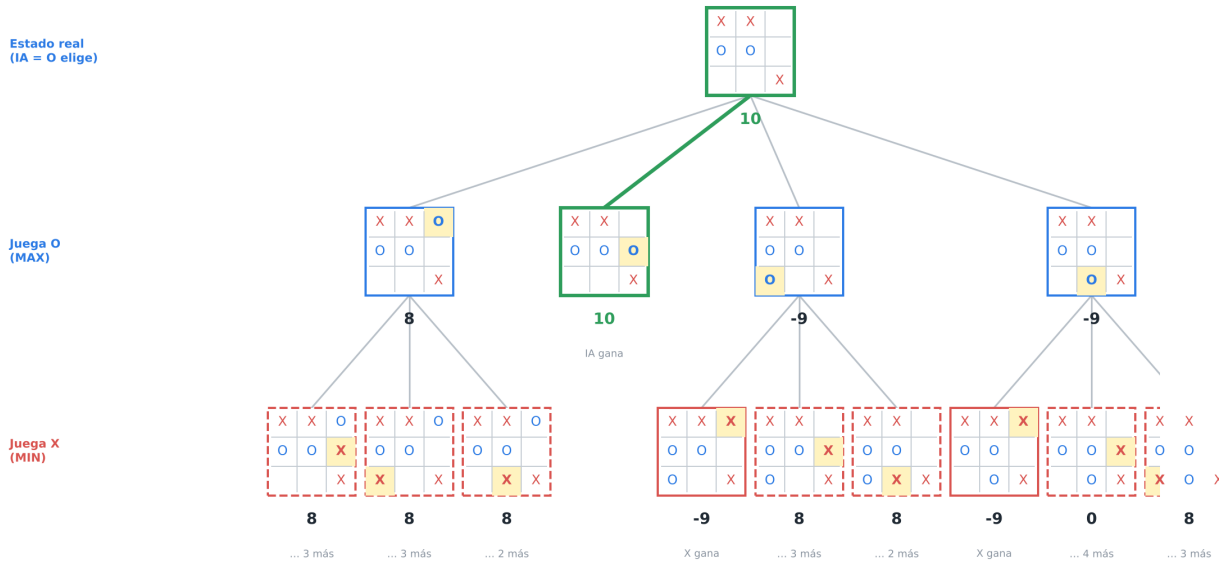


Figura 4: Árbol real de una posición (33 nodos). El número bajo cada tablero es el valor que devuelve `minimax`; en verde, la jugada elegida.

Cada nivel repite cuatro pasos: **bajar** (`make_move`), **recursión**, **deshacer** (`undo_move`) y **comparar** (`max` para la IA, `min` para el humano). El ajuste `10 - depth` / `-10 + depth` hace que la IA prefiera ganar rápido y perder lo más tarde posible.

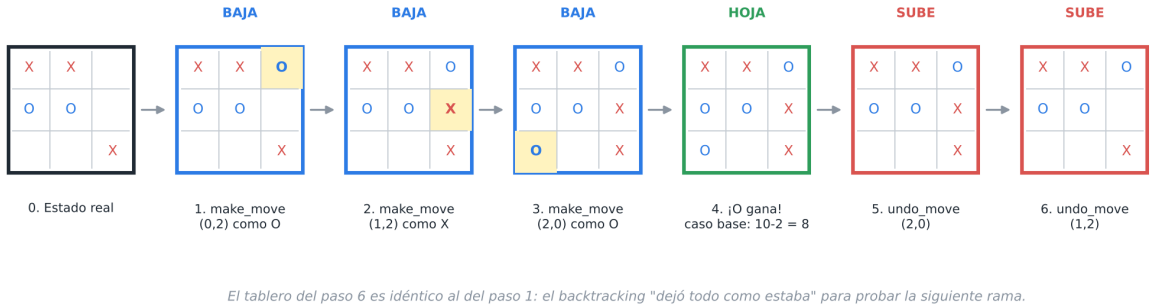


Figura 5: Backtracking: el tablero baja con `make_move` y vuelve idéntico con `undo_move`.

El árbol completo es grande aunque el juego sea pequeño:

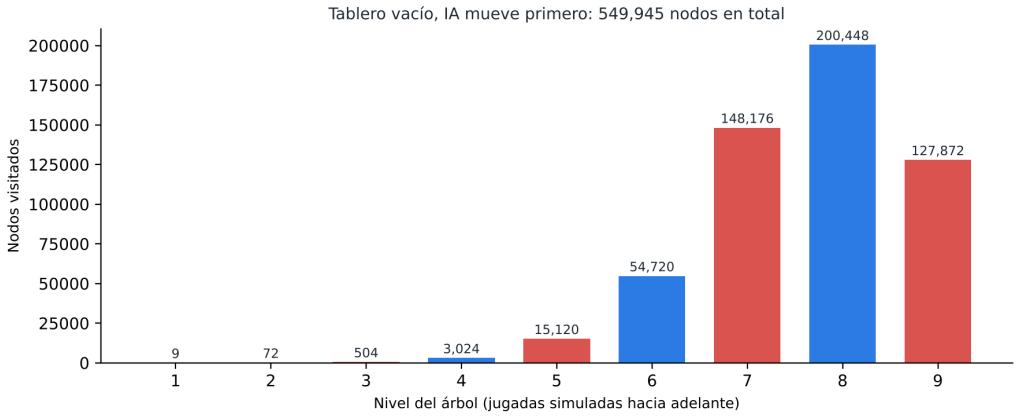


Figura 6: Nodos visitados por nivel cuando la IA abre en tablero vacío (549,945 en total).

6. Cómo se probó y qué quedó pendiente de tu lado

Pruebas realizadas sobre el código final (en un entorno de pruebas con Tkinter simulado, no con ojos humanos):

Prueba	Resultado
Minimax contra X probando todas las jugadas posibles	569 partidas, X nunca ganó; tablero intacto tras cada búsqueda
Interfaz en Humano vs Humano (victoria, clic en casilla ocupada, clic tras terminar)	Correcto
Modo Minimax: 200 partidas con X aleatorio	La IA nunca perdió; métricas visibles (p. ej. 59,704 nodos si X abre en esquina)
Modo ML: 100 partidas completas	Sin errores; muestra 0 nodos
Entrenamiento, reglas y gráfico (`python -m model.ml_agent`)	Correcto
Generación de dataset (con 12 partidas) y benchmark completo	Correcto
PEP8 con `pycodestyle`	De 224 avisos a **0**; `pyflakes` sin avisos

Lo que no pude comprobar: la generación de las 1,000 partidas completas (tarda varios minutos y el CSV ya existe) ni abrir la ventana en tu Windows. Pruébalo con `python main.py` antes de entregar.

Pendiente de tu lado:

1. Ejecutar `python main.py` y jugar una partida en cada modo.
2. Ejecutar `python comparativa_rendimiento.py` y actualizar la tabla de `reporte_comparativo.md` con tus números (ver 2.3).
3. Revisar los cambios con `git diff` y hacer tú el commit.

7. Tabla de nombres cambiados

Antes	Ahora	Archivo
`hacer_movimiento(f, c, j)`	`make_move((f, c), j)`	`game_model.py`
`deshacer_movimiento`	`undo_move`	`game_model.py`

Antes	Ahora	Archivo
obtener_movimientos_validos	get_available_moves	game_model.py
verificar_victoria	check_winner	game_model.py
rival_player	human_player	ml_agent.py
supervisor_minimax	minimax_ia (y minimax_rival)	ml_agent.py
dataset_records	registros	ml_agent.py
df	datos	ml_agent.py
X_train, X_test, y_train, y_test	x_entrenamiento, x_prueba, y_entrenamiento, y_prueba	ml_agent.py
model	modelo	ml_agent.py
MAPEO_FICHAS	VALOR_FICHA	ml_agent.py
COLUMNAS_CARACTERISTICAS / COLUMNA_OBJETIVO	COLUMNAS_CASILLAS / COLUMNA_JUGADA	ml_agent.py
prob_aleatorio_rival	prob_rival_aleatorio	ml_agent.py
agente_ia	agente_minimax	game_controller.py
vista.root.update()	vista.refrescar()	game_controller.py / gui_view.py
root	ventana	gui_view.py, main.py
lbl_mensaje, lbl_nodos, lbl_tiempo	etiqueta_mensaje, etiqueta_nodos, etiqueta_tiempo	gui_view.py
var_modo	modo_seleccionado	gui_view.py
frame_tablero, frame_metricas, panel_lateral	marco_tablero, marco_metricas, panel_control	gui_view.py
rb1, rb2, rb3	bucle sobre opciones_modo	gui_view.py
btn_reiniciar	boton_reiniciar	gui_view.py

Los nombres minimax, evaluate, get_best_move, nodes_evaluated, ai_player y human_player se **mantuvieron** porque son los del código de referencia del PDF.